

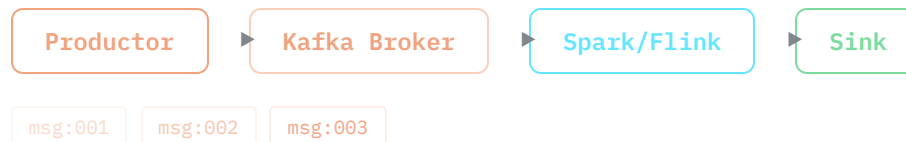
Streaming & Arquitectura de Eventos

Kafka · Spark Streaming · Apache Flink

Mensajería distribuida · Ventanas de tiempo · Watermarks

CLASES	DURACIÓN	FORMATO
10 & 11	2 horas	Teoría + Práctica

// PIPELINE EN TIEMPO REAL



AGENDA - 2 HORAS

00:00 → 00:50 · Clase 10: Kafka

00:50 → 01:00 · Break 🎧

01:00 → 01:50 · Clase 11: Streaming

01:50 → 02:00 · Presentación de Tarea

CLASE

10

MÓDULO 4 · INGESTA EN TIEMPO REAL

Ingesta de Datos con Apache Kafka

01 · TEORÍA

¿Qué es Apache Kafka?

Apache Kafka es una plataforma de **mensajería distribuida** de alto rendimiento diseñada para manejar flujos de datos en tiempo real a escala. Fue creado en LinkedIn y donado a la Apache Software Foundation.

Arquitectura de Eventos

En una **arquitectura orientada a eventos**, los componentes se comunican emitiendo y reaccionando a eventos. Kafka actúa como el *bus de comunicación central*.

①

Productor (Producer)

Aplicación que genera y publica mensajes/eventos a Kafka

②

Broker

Servidor Kafka que recibe, almacena y distribuye mensajes

③

Topic

Canal lógico donde se publican mensajes. Dividido en Partitions

Aplicación que lee mensajes de un topic para procesarlos

Tabla de Conceptos Clave

CONCEPTO	DESCRIPCIÓN
Offset	Identificador único de un mensaje dentro de una partición
Partition	División del Topic que permite paralelismo y escalabilidad
Consumer Group	Grupo de consumidores que distribuyen la lectura de particiones
Replication	Copias de las particiones en múltiples brokers para alta disponibilidad
Retention	Tiempo que Kafka guarda los mensajes (configurable, ej: 7 días)
ZooKeeper / KRaft	Coordinador del clúster (ZK legacy; KRaft es el nuevo modo nativo)

¿Por qué usar Kafka?



Alto Rendimiento

Millones de mensajes por segundo con latencia de milisegundos. Diseñado para I/O secuencial en disco.



Durabilidad

Los mensajes se persisten en disco y se replican. No se pierden datos ante fallos de nodos.



Desacoplamiento



Escalabilidad

DATA_ENG · CLASE 10-11

Independientes y tolerantes a fallos.

petabytes de datos.

CASO DE USO REAL

Uber usa Kafka para procesar más de un billón de mensajes por día: geolocalización de conductores, eventos de precio dinámico, detección de fraude y analítica en tiempo real.

02 · INSTALACIÓN

Levantando Kafka con Docker

La forma más rápida para desarrollo y clases es Docker Compose. No requiere instalar Java ni ZooKeeper manualmente.

PRE-REQUISITOS

Tener instalado Docker Desktop y Docker Compose. También Python 3.8+ para los ejemplos.

docker-compose.yml

```
# Paso 1: Crear archivo docker-compose.yml
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
```

DATA_ENG · CLASE 10-11

```
    - "9092:9092"
environment:
  KAFKA_BROKER_ID: 1
  KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

kafka-ui: # UI opcional para visualizar topics
  image: provectuslabs/kafka-ui:latest
  ports:
    - "8080:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
```

terminal

```
# Paso 2: Levantar servicios
docker-compose up -d

# Paso 3: Crear un Topic
docker exec -it kafka kafka-topics --create \
  --topic eventos-sensores \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 1

# Listar topics existentes
docker exec -it kafka kafka-topics --list \
  --bootstrap-server localhost:9092

# Instalar librería Python
pip install kafka-python
```

03 · PRÁCTICA

Ejemplo: Sistema de Sensores IoT

Simularemos sensores de temperatura enviando datos en tiempo real. Un productor genera lecturas cada segundo y un consumidor las procesa.



Productor (producer.py)

```
producer.py

import json, time, random
from kafka import KafkaProducer
from datetime import datetime

# Conectar al broker Kafka
producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

SENSORES = ['sensor-01', 'sensor-02', 'sensor-03']

while True:
    evento = {
        'sensor_id': random.choice(SENSORES),
        'temperatura': round(random.uniform(18.0, 45.0), 2),
        'humedad': round(random.uniform(30.0, 90.0), 2),
        'timestamp': datetime.now().isoformat()
    }

    producer.send('eventos-sensores', value=evento)
    print(f"✉ Enviado: {evento}")
    time.sleep(1)  # un evento por segundo
```



Consumidor (consumer.py)

DATA_ENG · CLASE 10-11

```
import json
from kafka import KafkaConsumer

# Suscribirse al topic
consumer = KafkaConsumer(
    'eventos-sensores',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='earliest', # desde el inicio
    group_id='grupo-monitoreo',
    value_deserializer=lambda m:
        json.loads(m.decode('utf-8'))
)

print("👂 Escuchando eventos...")

for mensaje in consumer:
    dato = mensaje.value
    temp = dato['temperatura']

    # Alerta si temperatura > 40°C
    alerta = "🔴 ALERTA" if temp > 40 else "🟢 OK"

    print(f"[{alerta}] {dato['sensor_id']}: "
          f"{temp}°C | offset={mensaje.offset}")
```

 CÓMO EJECUTAR EL EJEMPLO

1. Levantar Docker Compose → 2. Abrir Terminal A: `python producer.py` → 3. Abrir Terminal B: `python consumer.py` → 4. Observar los mensajes fluyendo en tiempo real. Visitar <http://localhost:8080> para ver la UI de Kafka.



04 · TEORÍA

Procesamiento de Streams

El procesamiento en tiempo real permite analizar datos mientras son generados, sin necesidad de almacenarlos primero. Clave en detección de fraude, alertas, dashboards en vivo.

Spark Streaming vs Apache Flink

Ventanas de Tiempo (Windows)

Las ventanas agrupan eventos en intervalos de tiempo para realizar agregaciones (suma, promedio, conteo). Son fundamentales en stream processing.

Marcas de Agua (Watermarks)

Los eventos pueden llegar **tarde o desordenados** en redes reales. Los watermarks son una señal que indica hasta qué punto en el tiempo el sistema asume que ya llegaron todos los eventos.

DATA_ENG · CLASE 10-11

aunque aún tolera eventos tardíos según la política configurada.

// VISUALIZACIÓN DE EVENTOS Y VENTANAS

Eventos →

e1:00

e1:30

e2:10

e2:45

e3:20

e3:55

e4:00*late

Tumbling Window (2 min)

Evento tardío (post-watermark)

05 · INSTALACIÓN

Configurar PySpark Streaming

Instalación PySpark

terminal

Instalar PySpark con soporte Kafka

```
pip install pyspark
```

Verificar versión (necesita Java 8 o 11)

```
java -version
```

```
python -c "import pyspark; print(pyspark.__version__)"
```

Variable de entorno para JAR de Kafka

```
export PYSARK_SUBMIT_ARGS="--packages \
    org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0 \
    pyspark-shell"
```

Flink con Docker (alternativo)

DATA_ENG · CLASE 10-11

```
services:
  jobmanager:
    image: flink:1.18-scala_2.12
    ports:
      - "8081:8081"
    command: jobmanager
    environment:
      JOB_MANAGER_RPC_ADDRESS: jobmanager

  taskmanager:
    image: flink:1.18-scala_2.12
    depends_on: [jobmanager]
    command: taskmanager
    environment:
      JOB_MANAGER_RPC_ADDRESS: jobmanager

# UI disponible en http://localhost:8081
```

06 · PRÁCTICA

Ejemplo: Promedio de Temperatura con PySpark Streaming

Conectar Spark al topic de Kafka que creamos en la Clase 10, calcular el promedio de temperatura por sensor cada 30 segundos usando una Tumbling Window.

spark_streaming.py

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    from_json, col, avg, window, current_timestamp
```

DATA_ENG · CLASE 10-11

```
# 1. Crear sesión Spark
spark = SparkSession.builder \
    .appName("MonitorTemperatura") \
    .config("spark.sql.shuffle.partitions", "2") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# 2. Esquema del JSON que envía el productor
schema = StructType() \
    .add("sensor_id", StringType()) \
    .add("temperatura", DoubleType()) \
    .add("humedad", DoubleType()) \
    .add("timestamp", StringType())

# 3. Leer stream desde Kafka
raw_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "eventos-sensores") \
    .option("startingOffsets", "latest") \
    .load()

# 4. Parsear JSON y agregar timestamp procesamiento
parsed = raw_stream \
    .selectExpr("CAST(value AS STRING) as json_str") \
    .select(from_json(col("json_str"), schema).alias("data")) \
    .select("data.*") \
    .withColumn("proc_time", current_timestamp())

# 5. Ventana tumbling de 30 segundos por sensor
resultado = parsed \
    .groupBy(
        window(col("proc_time"), "30 seconds"),
        col("sensor_id")
    ) \
    .agg(avg("temperatura").alias("temp_promedio"))

# 6. Escribir resultados en consola cada 30 segundos
query = resultado.writeStream \
    .outputMode("update") \
    .format("console") \
    .option("truncate", "false") \
```

DATA_ENG · CLASE 10-11

```
query.awaitTermination()
```

SALIDA ESPERADA EN CONSOLA

Cada 30 segundos verás una tabla como:

```
| window (start→end) | sensor_id | temp_promedio |  
| 12:00:00 → 12:00:30 | sensor-01 | 31.45 |
```

Watermarks en Flink (concepto clave)

flink_watermarks.py

```
from pyflink.datastream import StreamExecutionEnvironment  
from pyflink.common.watermark_strategy import WatermarkStrategy, Duration  
  
env = StreamExecutionEnvironment.get_execution_environment()  
  
# Estrategia de watermark: tolerar hasta 5 segundos de retraso  
watermark_strategy = WatermarkStrategy \  
    .for_bounded_out_of_orderness(Duration.of_seconds(5)) \  
    .with_timestamp_assigner(  
        # Extraer el timestamp del campo del evento  
        lambda event, _: event['event_time_ms']  
    )  
  
# Los eventos con más de 5s de retraso se DESCARTAN  
# Los eventos a tiempo se procesan en su ventana correcta  
stream = env \  
    .from_collection(datos_sensores) \  
    .assign_timestamps_and_watermarks(watermark_strategy)  
  
print("Watermark configurado. Tolerancia: 5 segundos.")
```

07 · RESUMEN

Lo que aprendimos hoy



CLASE 10 · 00:00-00:20

Arquitectura Kafka

Topics, Partitions, Brokers, Consumer Groups. Modelo publish-subscribe. Por qué Kafka escala a millones de eventos/s.



CLASE 10 · 00:20-00:50

Práctica: Producer & Consumer en Python

Levantamos Kafka con Docker, creamos el topic `eventos-sensores`, implementamos producer y consumer con kafka-python.



CLASE 11 · 01:00-01:25

Teoría Streaming: Spark vs Flink

Diferencias clave. Ventanas Tumbling, Sliding y Session. Event Time vs Processing Time. Watermarks para eventos tardíos.



CLASE 11 · 01:25-01:50

Práctica: PySpark Structured Streaming + Kafka

Conectamos Spark al topic de Kafka. Parseamos JSON, aplicamos ventana tumbling de 30s y calculamos promedio por sensor.



CLASE 11 · 01:50-02:00

Asignación de Tarea Grupal

Pipeline completo Kafka → Spark Streaming con múltiples fuentes, ventanas y persistencia en base de datos.

08 · TAREA GRUPAL

Pipeline de Monitoreo en Tiempo Real

PROYECTO GRUPAL · 3-4 INTEGRANTES

Construir un pipeline completo de datos en tiempo real que simule el monitoreo de una planta industrial con múltiples tipos de sensores. El sistema debe ingestar datos con Kafka y procesarlos con Spark Streaming, generando alertas y estadísticas en tiempo real.

ENTREGA

1 semana

GRUPOS

3-4 personas

MODALIDAD

Repo GitHub + Demo



Descripción del Escenario

Una fábrica tiene 3 líneas de producción, cada una con sensores de temperatura, presión y vibración enviando lecturas cada segundo. El sistema debe:

- Recibir eventos en tiempo real por Kafka (1 topic por tipo de sensor)
- Procesar con Spark Streaming usando ventanas de tiempo
- Detectar anomalías y generar alertas automáticas
- Guardar estadísticas en archivo CSV o base de datos SQLite



Arquitectura Esperada

producer_temp.py

producer_presion.py

topic: temperatura

topic: presion

 Entregables Requeridos

ENTREGABLE 01

Productores Kafka

Al menos 2 scripts productores simulando distintos tipos de sensores con datos aleatorios realistas

ENTREGABLE 02

Processor Spark

Script PySpark que lea de Kafka, aplique ventana tumbling de 1 minuto y calcule min/max/promedio

ENTREGABLE 03

Sistema de Alertas

Lógica para detectar y registrar anomalías: temperatura > umbral, presión fuera de rango, etc.

ENTREGABLE 04

Persistencia

Los resultados procesados se guardan en CSV o SQLite con timestamp, sensor_id y estadísticas

ENTREGABLE 05

README + docker-compose

Documentación para levantar todo el sistema con un solo comando. Instrucciones claras paso a paso

ENTREGABLE 06

Demo en Clase (5 min)

Demostración en vivo del pipeline funcionando: datos fluyendo, ventanas calculándose, alertas disparándose

 Criterios de Evaluación

CRITERIO	DESCRIPCIÓN	PESO
Pipeline funcional	Kafka → Spark → Sink todo conectado y operando	30%
Ventanas de tiempo	Uso correcto de al menos 2 tipos de ventana distintos	20%
Detección de anomalías	Sistema de alertas con al menos 2 condiciones configurables	20%
Código y documentación	Código limpio, comentado, README completo	15%
Demo y presentación	Claridad en la explicación, todos los miembros participan	15%

DATA_ENG · CLASE 10-11

configurados · +0.2 Dashboard visual simple con Streamlit o Grafana · +0.1 Despliegue completo en Docker Compose con todos los servicios

DATA_ENG Clases 10 & 11 · Apache Kafka · Spark Streaming · Apache Flink

Recursos: kafka.apache.org · spark.apache.org · flink.apache.org